软工

题型

1. 单项选择题 20（20*1）
2. 简答题 20（4*5）
3. 用例分析设计 UML 20（4*5）
4. 设计类 20（2*10）
5. 综合、测试 20（2*10）

Introduction

Software?

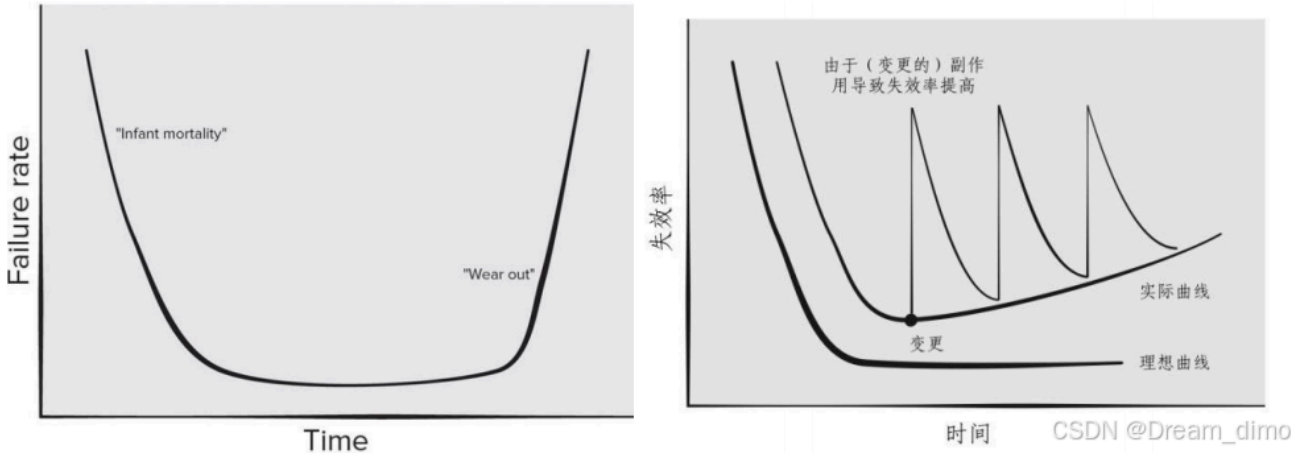
- Programs (instructions)
- Data Structure (data structures)
- Document (descriptive information)

> Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information and (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.
> 软件是 :(1)指令的集合(计算机程序)，通过执行这些指令可以满足预期的特性、功能和性能需求 ;(2)数据结构，使得程序可以合理利用信息 ;(3)软件描述信息，它以硬拷贝和虚拟形式存在，用来描述程序的操作和使用。

Software Characteristics

(The differences between software and hardware)

- Software is developed,not manufactured. 软件是开发出来的，而不是制造出来的。



- Software does not wear out, but it can deteriorate. 软件不会磨损，但会退化。
- Most software is custom build, not assembled out of components. 大多数软件是定制构建的，而不是由组件组装而成的。

> **硬件失效率曲线（左图）：**
> 硬件的失效率随着时间的推移表现为一个“浴缸曲线”。
> 初始阶段的高失效率（称为“婴儿死亡率”），这是由于初期故障较多。
> 中期失效率较低且稳定。
> 最后阶段失效率再次升高，表示硬件开始“磨损”并逐渐失效。
> **软件失效率曲线（右图）：**
> 软件的失效率在初始阶段较高，主要由于早期的缺陷和错误。
> 随着时间推移，失效率下降，但并不会像硬件那样达到一个恒定的低值，而是随着修改和更新（如图中标注的“更改”）可能再次升高。
> 这种波动的失效率是由于每次修改可能引入新的错误，导致失效率上升，随后再下降到一个较低的水平，但不会完全稳定。

Software is deteriorating 退化(Why?)

> 在完整的生命周期里，软件将会面临变更，每次变更都可能引入新的错误，使得失效率像“实际曲线”那样陡然上升。在曲线回到最初的稳定失效率状态前，新的变更会引起曲线又一次上升。就这样，最小的失效率点沿类似于斜线的形状逐渐上升，可以说，不断的变更是软件退化的根本原因。

Software engineering

- The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.
- The study of approaches as in above.

> 软件工程是 :(1)将系统化的、规范的、可量化的方法应用于软件的开发、运行和维护，即将工程化方法应用于软件 ;(2)对(1)中所述方法的研究。

The Essence of Practice 实践的精髓

- Understand the problem(communication and analysis).
- Plan a solution (modeling and software design).
- Carry out the plan(code generation).
- Examine the result for accuracy(testing and quality assurance).

> 1. 理解问题(沟通和分析)
> 2. 策划解决方案(建模和软件设计)
> 3. 实施计划(代码生成)。
> 4. 检查结果的正确性(测试和质量保证)

Software process

It's a framework for the activities, actions, and tasks that are required to build high quality software.

> 是构建高质量软件时所需的一系列活动、动作和任务的集合。

🔔
- 软件必须适应新的计算环境或技术
- 软件必须增强以实现新的业务需求
- 软件必须扩展以与其他更现代的系统或数据库进行互操作
- 软件必须重新架构以在网络环境中具有可行性

Software Process

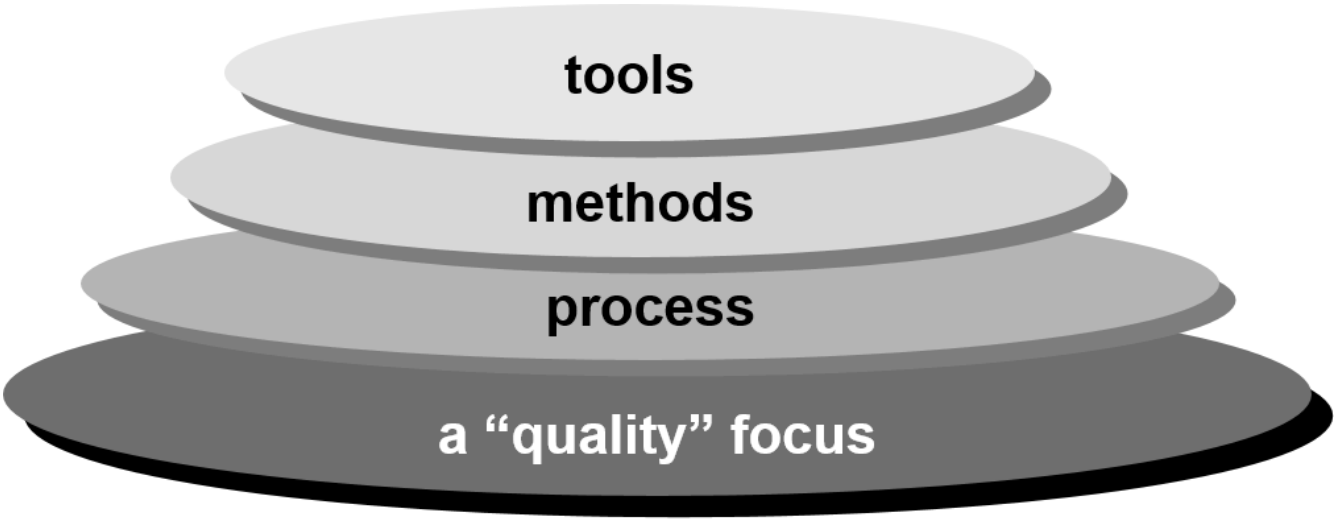
What is software process?

Collection of activities, actions & tasks

> 一系列活动、动作和任务的集合。

Software Engineering Layers

软件工程是一种层次化的技术； 软件工程的基础是过程；软件工程的根基在于“质量关注点”



> 工具、方法、过程、质量关注点

Generic Software Engineering Framework Activities

- Communication 沟通
- Planning 策划
- Modeling 建模
  - Analysis of requirements 需求分析
  - Design 设计
- Construction 构建
  - Code generation 代码生成
  - Testing 测试
- Deployment 部署

> Difference process models for SE proposed, all process models define a set of framework activities, a collection of tasks that conducted to accomplish each activity, work products produced as a consequence of the tasks, and a set of umbrella activities that span the entire process.
> 针对软件工程提出了不同的过程模型，所有过程模型都定义了一组框架活动、一系列为完成每个活动而进行的任务、因这些任务产生的工作产品以及贯穿整个过程的一组通用活动。

Umbrella Activities 普适性活动

- 软件项目跟踪和控制
- 风险管理
- 软件质量保证
- 技术评审
- 测量

- 软件配置管理
- 可复用管理
- 工作产品的准备和生产

Process Flow 过程流

- linear process flow **线性过程流**：从沟通到部署顺序执行五个框架活动
- iterative process flow **迭代过程流：**在执行下一个活动前，重复执行之前的一个或多个活动
- evolutionary process flow **演化过程流：**采用循环的方式执行各个活动
- parallel process flow **并行过程流**：将一个或多个活动与其他活动并行执行

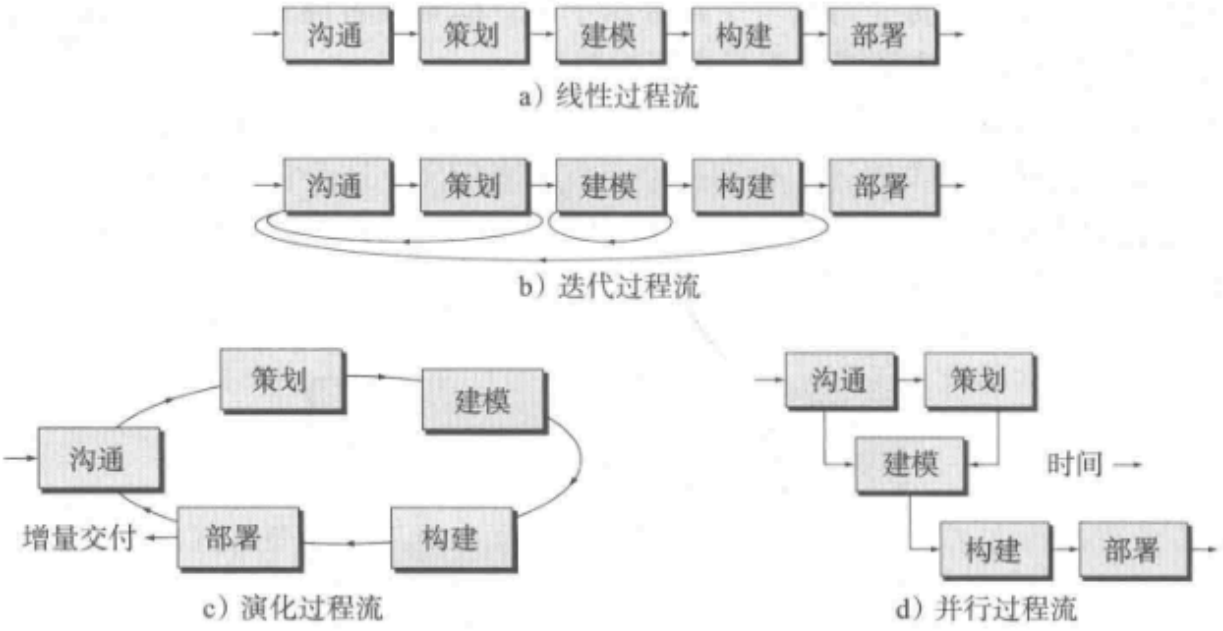


图 3-2 过程流

Process Assessment and Improvement 流程评估和改进

- Standard CMMI Assessment Method for Process Improvement (SCAMPI)： 用于过程改进的 CMMI 标准评估方法 .
- CMM-Based Appraisal for Internal Process Improvement (CBA IPI)：用于组织内部过程改进的 CMM 评估
- SPICE—The SPICE (ISO/IEC15504)：软件过程改进和能力测定（software process improvement and capability determination）
- ISO 9001:2000 for Software：国际标准化组织的软件开发标准
- GB 国标

Process Models 过程模型

- *Waterfall Model (Linear Sequential Model)*
- *Incremental Model (core product +waterfall)*
- *Prototyping Model (evolutionary model, requirements fuzzy, throw-away)*
- *Spiral Model(evolutionary models + Prototyping + waterfall, Suitable for large,complex projects.)*
- *Unified Process Model(an iterative and incremental evelopment process, UML*
- *use-case driven, architecture-centric, Risk Focused)*
- *evolutionary models (the iterative nature, accomodate change)*

惯用过程模型

瀑布模型（Waterfall Model ）

基于软件生命周期的线性工作模型，提出了系统、顺序地开发软件。从用户规格说明书开始，通过策划、建模、构建、部署，提供完整软件和持续的软件支持

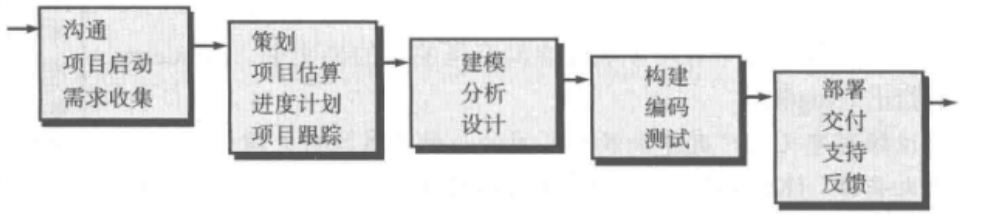


图 4-1 瀑布模型

Pros

- It is easy to understand and plan.
- It works for well-understood small projects.
- Analysis and testing are straightforward.

Cons

- It does not accommodate change well.
- Testing occurs late in the process.
- Customer approval is at the end.

> **优点**
> 易于理解和规划
> 适用于已充分理解的小型项目
> 分析和测试过程简单明了

> **缺点**
> 不善于应对变化
> 测试在流程的后期进行
> 客户的认可在最后阶段才获得

增量模型（Incremental Model ）

增量模型综合了第 3 章讨论的线性过程流和并行过程流的特征。随着时间的推移，增量模型在每个阶段都运用线性序列。每个线性序列生产出软件的可交付增量。

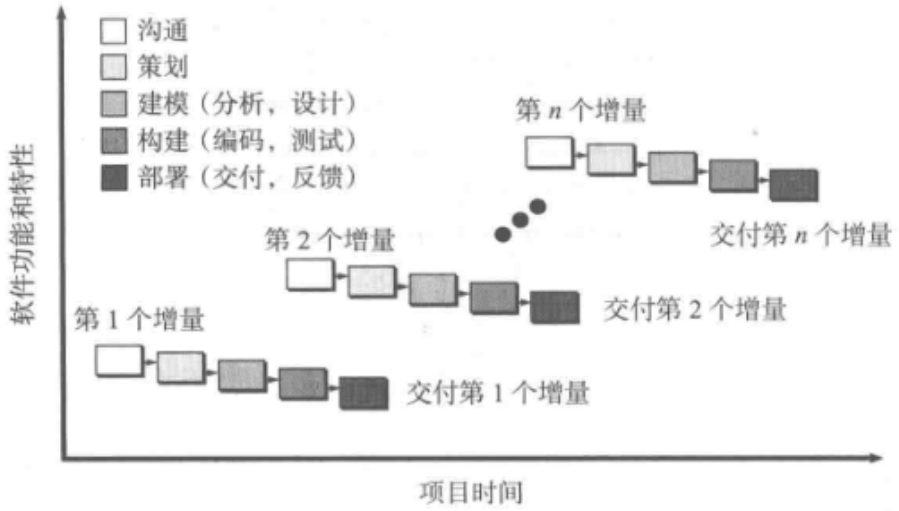


图 4-3　增量模型

> **优点**
> 克服人手不足：在业务截止日期前无法配齐全部人员时非常有用。
> 节约时间
> 线性（每个增量按照瀑布模型进行管理）+ 并行处理。
> 可交付的运行版本：每个增量都是可交付的运行版本，第一个增量往往是核心产品（core product）。
> **缺点**
> 关注每个增量的运行产品交付：容易忽视早期的增量。

演化过程模型

原型模型（Prototyping Model）

原型开发范型(图 4-4)开始于沟通。软件开发人员和其他利益相关者进行会晤，定义软件的整体目标，明确已知的需求，并大致勾画出以后再进一步定义的东西。然后迅速策划一个原型开发迭代并进行建模(以“快速设计”的方式)。快速设计要集中在那些最终用户能够看到的方面(比如人机接口布局或者输出显示格式)。快速设计产生了一个原型。对原型进行部署，然后由利益相关者进行评估。根据利益相关者的反馈信息，进一步精炼软件的需求。在原型系统不断调整以满足各种利益相关者需求的过程中，采用迭代技术，同时也使开发者逐步清楚用户的需求。

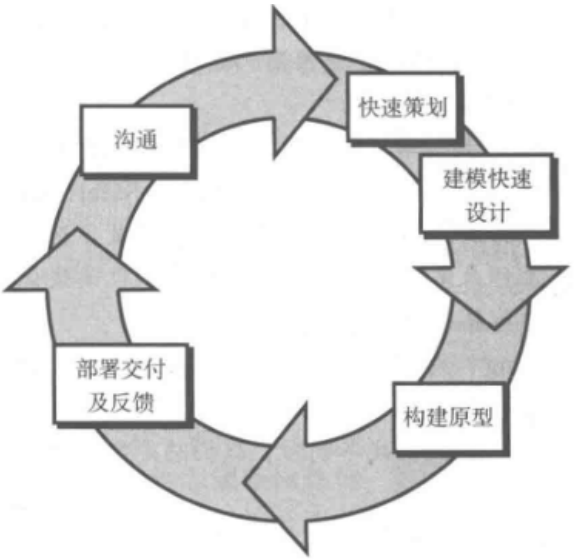


图 4-4　原型开发范型

Pros

- Reduced impact of requirement changes.
- Customer is involved early and often.
- Works well for small projects.
- Reduced likelihood of product rejection.

Cons

- Customer involvement may cause delays.
- Temptation to “ship” a prototype.
- Work lost in a throwaway prototype.
- Hard to plan and manage.

> **优点**
> 减少需求变更的影响。
> 客户早期且频繁地参与。
> 适合小型项目。
> 减少产品被拒绝的可能性。
> **缺点**
> 客户的参与可能导致延误。
> 可能会倾向于“发布”一个原型。
> 在废弃的原型上浪费工作。
> 难以规划和管理。

螺旋模型（Spiral Model）

螺旋模型是一种演化过程模型，它结合了原型的迭代性质和瀑布模型的系统性和可控性，注重风险控制（里程碑 milestone），适合大型项目开发

螺旋模型两个显著特点：

- 采用循环的方式逐步加深系统定义和实现的深度
- 确定一系列里程碑，确保利益相关者都支持可行的和令人满意的系统解决方案

运用螺旋模型，把软件开发为一系列演进版本，早期的迭代中，软件可能是一个理论模型或原型，后来的迭代，才会产生一系列逐渐完善的系统版本

在每一次迭代的过程中，都要考虑风险、标记里程碑

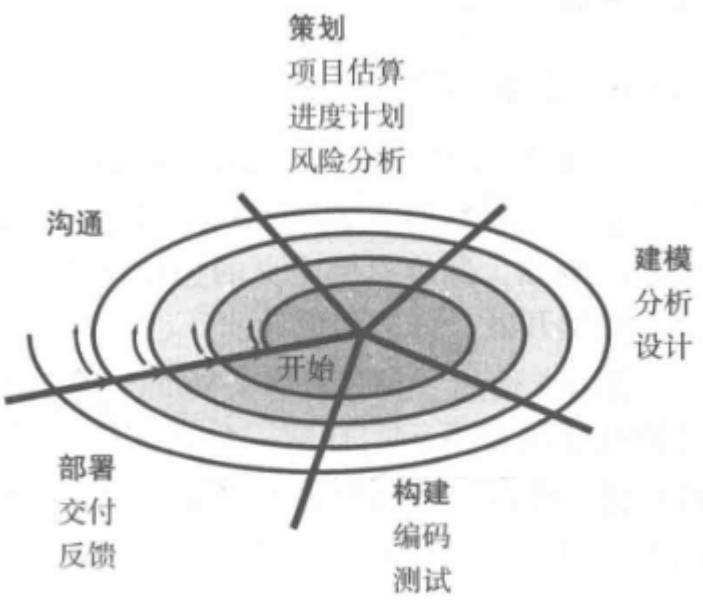


图 4-5　典型的螺旋模型

Pros

- Continuous customer involvement.
- Development risks are managed.
- Suitable for large, complex projects.
- It works well for extensible products.

Cons

- Risk analysis failures can doom the project.
- Project may be hard to manage.
- Requires an expert development team.

> **优点**
> 持续的客户参与。
> 管理开发风险。
> 适用于大型、复杂的项目。
> 适合可扩展的产品。
> **缺点**

> 风险分析失败可能会导致项目失败。
> 项目管理可能会很困难。
> 需要一个经验丰富的开发团队。

并发模型

统一过程模型（Unified Process Model）

The Unified Modeling Language (UML) is “a standard language for writing software blueprints. UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system”. In other words, just as building architects create blueprints to be used by a construction company, software architects create UML diagrams to help software developers build the software. If you understand the vocabulary of UML (the diagrams’ pictorial elements and their meanings), you can much more easily understand and specify a system and explain the design of that system to others.

> 统一建模语言（UML）是“一种用于编写软件蓝图的标准语言。UML 可以用于可视化、指定、构建和记录软件密集型系统的工件”。换句话说，就像建筑师创建蓝图以供施工公司使用一样，软件架构师创建 UML 图以帮助软件开发人员构建软件。如果您了解 UML 的词汇（图表的图形元素及其含义），您将更容易理解和指定系统，并向他人解释该系统的设计。

- The Unified Process is an iterative and incremental development process. The Elaboration, Construction and Transition phases are divided into a series of time-boxed iterations. (The Inception phase may also be divided into iterations for a large project.) Each iteration results in an *increment*, which is a release of the system that contains added or improved functionality compared with the previous release. provides a process and methodological approach for defining, specifying, designing, and constructing *aspects*
- use-case driven （用例驱动）
- architecture-centric （软件架构为核心）
- Risk Focused （关注风险）
- Inception/Elaboration/Construction/Transition（/production）

> 统一过程是一种迭代和增量的开发过程。阐述、构建和转换阶段被分为一系列时间固定的迭代。（对于大型项目，初始阶段也可能被分为多个迭代。）每次迭代都会产生一个*增量*，即包含相对于之前版本增加或改进功能的系统发布版本。它提供了一个定义、指定、设计和构建*方面*的过程和方法。
> 用例驱动
> 软件架构为核心

> 关注风险
> 起始 / 细化 / 构建 / 转换（/ 生产）

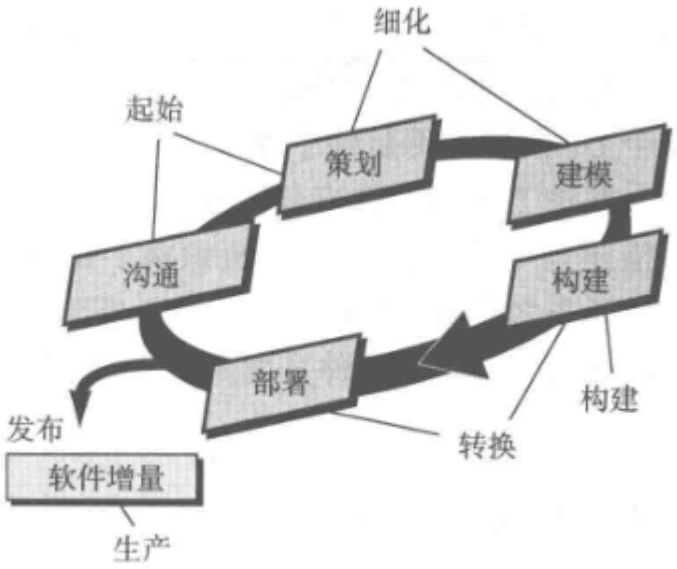


图 4-7　统一过程

Pros

- Quality documentation emphasized.
- Continuous customer involvement.
- Accommodates requirements changes.
- Works well for maintenance projects.

Cons

- Use cases are not always precise.
- Tricky software increment integration.
- Overlapping phases can cause problems.
- Requires expert development team.

> **优点**
> 强调高质量的文档。
> 持续的客户参与。
> 能够适应需求变化。
> 适用于维护项目。
> **缺点**
> 用例不总是精确的。
> 软件增量集成复杂。
> 阶段重叠可能引发问题。
> 需要专家开发团队。

敏捷开发（Agile）

What is agile? 快速、增量（迭代）

- Rapid deliver products（not care intermediate work products)
- Effective (rapid and adaptive) response to change
- Customers join the team（eliminate “us and them”), Effective communication among all stakeholders
- Project plan must be flexible
- Organizing a team so that it is in control of the work performed(自组织) ,communicate facilely

> 快速交付产品（不关注中间工作产品）
> 有效（快速且适应性强）地应对变化
> 客户加入团队（消除“我们和他们”的对立），所有利益相关者之间有效沟通
> 项目计划必须灵活
> 组织团队，使其能够控制所执行的工作（自组织），轻松沟通

The Manifesto for Agile（敏捷宣言）

- Individuals and interaction OVER processes and tools

  个人与交互　　过程和工具

- Working software OVER comprehensive documentation

  可运行软件　　繁复文档

- Customer collaboration OVER contract negotiation

  客户合作　　合同谈判

- responding to change OVER following a plan

  响应变更　　遵循计划

Extreme Programming(XP): planning, design, coding, test

- User stories
- KIS
- Pair-programming
- Refactoring
- Continuous integration
- Incremental delivery

> 用户故事
> 保持简单（KIS：Keep It Simple）
> 结对编程
> 重构
> 持续集成
> 增量交付

🔔 结对编程：

- XP 建议两个人在同一台计算机工作站上一起工作，为一个功能创建代码
- 这提供了一种实时解决问题的机制（两个头脑通常比一个头脑好）
- 和实时质量保证（代码在创建时进行审查）
- 它还使开发人员专注于手头的问题

- XP Planning – Begins with user stories, team estimates cost, stories grouped into increments, commitment made on delivery date, computer project velocity.
- XP Design – Follows KIS principle, encourages use of CRC cards, design prototypes, and refactoring.
- XP Coding – construct unit tests before coding, uses pair.
- XP Testing – unit tests executed daily, acceptance tests define by customer.

> **XP 计划** – 从用户故事开始，团队估算成本，将故事分组成增量，承诺交付日期，计算项目速度。
> **XP 设计** – 遵循 KIS 原则，鼓励使用 CRC 卡片、设计原型和重构。
> **XP 编码** – 在编码前编写单元测试，使用结对编程。
> **XP 测试** – 每天执行单元测试，由客户定义验收测试。

Pros

- Emphasizes customer involvement.
- Establishes rational plans and schedules.
- High developer commitment to the project.
- Reduced likelihood of product rejection.

Cons

- Temptation to “ship” a prototype.
- Requires frequent meetings about increasing costs.
- Allows for excessive changes.
- Depends on highly skilled team members.

> **优点**
> 强调客户参与。
> 制定合理的计划和时间表。
> 开发人员对项目有高度承诺。
> 降低产品被拒绝的可能性。
> **缺点**
> 可能会倾向于“发布”一个原型。
> 需要频繁召开增加成本的会议。
> 允许过多的变更。
> 依赖高技能的团队成员。

Scrum (Requirements, Analysis, Design, Evolution, Delivery)

- Development work is partitioned into “packets(小包)”
- Testing and documentation are on-going as the product is constructed
- Work occurs in “sprints(冲刺)” and is derived from a “backlog（待定项，积压的工作)” of existing requirements
- Meetings are very short and sometimes conducted without chairs
- “demos（演示)” are delivered to the customer with the time-box allocated

> 开发工作被划分为“包”（小包）
> 在产品构建过程中，测试和文档编写是持续进行的
> 工作在“冲刺”中进行，源自于现有需求的“待定项”（积压的工作）
> 会议时间很短，有时甚至没有椅子
> “演示”会在分配的时间段内交付给客户

- Backlog Refinement Meeting Developers work with stakeholders to create product backlog.
- Sprint Planning Meeting Backlog partitioned into “sprints” derived from backlog and next sprint defined.
- Daily Scrum Meeting Team members synchronize their activities and plan work day (15 minutes max).
- Sprint Review Prototype “demos” are delivered to the stakeholders for approval or rejection.
- Sprint Retrospective After sprint is complete, team considers what went well and what needs improvement.

> **待办项梳理会议**：开发人员与利益相关者合作创建产品待办项。
> **冲刺计划会议**：将待办项划分为“冲刺”，从待办项中导出并定义下一个冲刺。

> **每日站会**：团队成员同步他们的活动并计划工作日（最多 15 分钟）。
> **冲刺评审**：向利益相关者交付原型“演示”以供批准或拒绝。
> **冲刺回顾**：冲刺完成后，团队会考虑哪些做得好以及需要改进的地方。

Pros

- Product owner sets priorities.
- Team owns decision making.
- Documentation is lightweight.
- Supports frequent updating.

Cons

- Difficult to control the cost of changes.
- May not be suitable for large teams.
- Requires expert team members.

> **优点**
> 产品负责人设定优先级。
> 团队自主决策。
> 文档精简。
> 支持频繁更新。
> **缺点**
> 难以控制变更成本。
> 可能不适合大型团队。
> 需要专家级团队成员。

Kanban（看板）

DevOps(Development &Operations) 开发和运营维护

Requirement

The definition of requirements engineering

- RE is a major process action starting the communication activity continues into modeling activity
- RE is a bridge to design and construction

> 需求工程(RequirementEngineering，RE)是指致力于不断理解需求的大量任务和技术。
> 需求工程在设计和构建之间建立起联系的桥梁。

Inception, Elicitation, Elaboration, Negotiation, Specification, Validation, Requirements Management

> 需求工程包括七项明确的任务：起始，获取，细化，协商，规格说明，确认和管理。注意，这些需求工作中的一些任务会并行发生，并且要全部适应项目的要求。

Inception（起始）—establish basic understanding of the project…

📌 establish basic understanding of the problem

the people who want a solution

the nature of the solution that is desired, and

the effectiveness of preliminary communication and collaboration between the customer and the developer

建立对问题的基本理解

想要解决方案的人

所需解决方案的性质，以及

客户和开发者之间初步沟通与协作的有效性

市场分析，粗略可行性分析(在需求之前)

Elicitation（收集）—elicit requirements from all stakeholders

📌 系统或产品的目标是什么？想要实现什么？系统和产品如何满足业务要求？最终系统或产品如何用于日常工作?

In fact, elicit requirements is very hard because of the following problems:

Problems of scope　　范围问题（系统边界不清楚）

Problems of understanding 理解问题（客户并不能完全确定需要什么）

Problems of volatility　　易变问题(需求随时间变化)

In order to overcome these problems, you must approach requirements gathering in an organized manner

Elaboration（精化）—create an analysis model that identifies data, function and behavioral requirements

📌 The information obtained from the customer during inception and elicitation is expanded and refined during elaboration

在起始和导出阶段获得的信息在精化阶段进行扩展和提炼，该任务集中于开发一个精确的需求模型（chapter6-7），用以说明软件的功能、特征等

Requirement modeling: Define classes, use-cases, various diagrams…

Negotiation（协商）—agree on a deliverable system that is realistic for developers and customers

📌 Reconcile conflicts between the users and developers

QFD[quality function deployment Normal /Expected/Exciting requirements

协调用户与开发者之间的冲突

QFD（质量功能部署）：正常 / 预期 / 令人兴奋的需求

Specification（规格说明）—document, combining natural language descriptions, graphic models or usage scenarios (use-cases)…

📌 A written document

A set of models

A formal mathematical

A collection of user scenarios (use-cases)

A prototype

Validation（确认）—a review mechanism that looks for

📌 errors in content or interpretation(解释)

areas where clarification（说明、澄清） may be required

missing information

Inconsistencies 不一致 - 矛盾 (a major problem when large products or systems are engineered)

conflicting or unrealistic (unachievable) requirements.

Requirements management（需求管理）

理解客户需要什么、分析要求、评估可行性、协商合理方案、无歧义地详细说明方案、确认规格说明、管理需求以及将这些需求转化为可行性运行系统。

需求工程通过执行上述七个活动来收集、分析、理解需求。

Requirements elicitation (=requirements gathering) combines elements of problem solving, elaboration, negotiation, and specification. In order to encourage a collaboration, team-oriented approach to requirements gathering, stakeholders work together to identify the problem, propose elements of the solution, negotiate different approaches and specify a preliminary set of solution requirements.

> 需求获取(又称为需求收集)将问题求解、细化、协商和规格说明等方面的元素结合在一起。为了鼓励合作，一个包括利益相关者和开发人员的团队共同完成如下任务：确认问题，为解决方案的相关元素提供建议，商讨不同的方法并描述初步的需求解决方案。

Elicitation Work Products 获取工作产品

A statement of need and feasibility.

A bounded statement of scope for the system or product.

A list of customers, users, and other stakeholders who participated in requirements elicitation

A description of the system’ s technical environment.

A list of requirements (preferably organized by function) and the domain constraints that apply to each.

A set of usage scenarios that provide insight into the use of the system or product under different operating conditions.

Any prototypes developed to better define requirements.

Requirement Specification 需求规格

🔔 软件需求规格（SRS）

- 技术需求列表
- SRS 描述了功能、接口和系统约束

> 根据将要构建的系统或产品规模的不同，需求获取后产生的工作产品也不同。对于大多数系统而言，工作产品包括：
> 要求和可行性陈述；
> 系统或产品范围的界限说明；

> 参与需求获取的客户、用户和其他相关利益者的名单；
> 系统技术环境的说明；
> 需求列表(最好按照功能加以组织)以及每个需求适用的领域限制；
> 一系列使用场景，有助于深入了解系统或产品在不同运行环境下的使用；
> 任何能够更好地定义需求的原型。
> 所有参与需求获取的人员需要评审以上的每一个工作产品。

Quality Function Deployment (QFD) 质量功能部署

QFD 是一种将客户要求转化成软件技术需求的技术。

Three types of requirements

- Normal requirements
- Expected requirements
- Exciting requirements

QFD 的目的是最大限度地让客户从软件工程中感到满意

ER 图

用例

An use-case describes the actions system takes to deliver something of value to the actor.

> 用例是对一组动作序列的抽象描述，就是系统功能描述

Scenarios —often called use cases, provide a description of how the system will be used.

> 情景——通常称为用例，提供了系统使用方式的描述。

举例：编写用例图（用例图是指由参与者 Actor、用例 Use Case，边界以及它们之间的关系构成的用于描述系统功能的视图，包括出错处理）

举例：编写用例场景（用例描述：包括：条件、场景、意外处理等，参考 PPT)

Use-case：InitiateMonitoring

Primary actor: Homeowner

Goal in context: To set the system to monitor sensors when the homeowner leaves the house or remains inside

Precondition： System has been programmed for a password and to recognize various sensors

Trigger: The homeowner decides to “set” the system, i.e., to turn on the alarm functions

Scenario:

1.Observes control panel;

2.Enters password;

3.Selects “stay” or “away”;

4.Observes read alarm light to indicate that the system has been armed.

Exception:

1.Control panel is not ready： homeowner checks all sensors…

2.Password is incorrect: homeowner reenters correct password;

3.Password not recognized: monitoring and response subsystem must be contacted to reprogram password;

4.Stay is selected: beep twice & stay light is lit; perimeter sensors are activated;

5.Away is selected: beep three times & away light is lit; perimeter sensors are activated;

用例：初始化监控。
主要参与者：房主。
目标：在房主离开住宅或留在房间时，设置系统以监控传感器。
前提条件：系统支持密码输入和传感器识别功能。
触发器：房主决定“设置”系统，即打开警报功能。
场景：
1. 房主：观察控制面板。
2. 房主：输入密码。
3. 房主：选择“stay”或“away”。
4. 房主：观察红色报警灯显示 SafeHome 已经被打开。
异常：
1. 控制面板没有准备就绪：房主检查所有的传感器，确定哪些是开着的（即门窗是开着的），并将其关闭。
2. 密码不正确（控制面板鸣叫一声）：房主重新输入正确的密码。
3. 密码不识别：必须对监控和响应子系统重新设置密码。
4. 选择 stay：控制面板鸣叫两声并且 stay 灯点亮；激活边界传感器。
5. 选择 away：控制面板鸣叫三声并且 away 灯点亮；激活所有传感器。

Objectives of Requirements Model 需求建模的目标

To describe what the customer requires 描述客户需要什么

To establish a basis for the creation of a software design 为软件设计奠定基础

To define a set of requirements that can be validated once the software is built 定义在软件完成后可以被确认的一组需求

Rules of Thumb 需求分析建模原则

The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high. Don’ t get bogged down in details(不要陷入细节)

Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function and behavior of the system. （增加对软件需求主体的理解）

Delay consideration of infrastructure and other non-functional models until design. (有些非功能的模型可以延后到设计阶段完成)

Minimize coupling throughout the system. (低耦合)

Be certain that the analysis model provides value to all stakeholders. (确认需求模型对所有利益相关者带来价值)

Keep the model as simple as it can be. （简洁）

> 模型应关注在问题域或业务域内可见的需求，抽象的级别应该相对高一些。*“不要陷入细节”，即不要试图解释系统将如何工作。*
> 需求模型的每个元素都应能增加对软件需求的整体理解，并提供对信息域、功能和系统行为的深入理解。
> 关于基础结构和其他非功能的模型应推延到设计阶段再考虑。*例如，可能需要一个数据库，但是只有在已经完成问题域分析之后才应考虑实现数据库所必需的类、访问数据库所需的功能以及使用时所表现出的行为。*
> 最小化整个系统内的关联。*表现类和功能之间的联系非常重要，但是，如果“互联”的层次非常高，则应该想办法减少互联。*
> 确认需求模型为所有利益相关者都带来价值。*对模型来说，每个客户都有自己的使用目的。例如，业务人员将使用模型确认需求，设计人员将使用型作为设计的基础，质量保证人员将使用模型帮助规划验收测试。*
> 尽可能保持模型简洁。*如果没有提供新的信息，就不要添加附加图表；如果一个简单列表够用，就不要使用复杂的表示方法。*

Requirements Modeling 需求建模

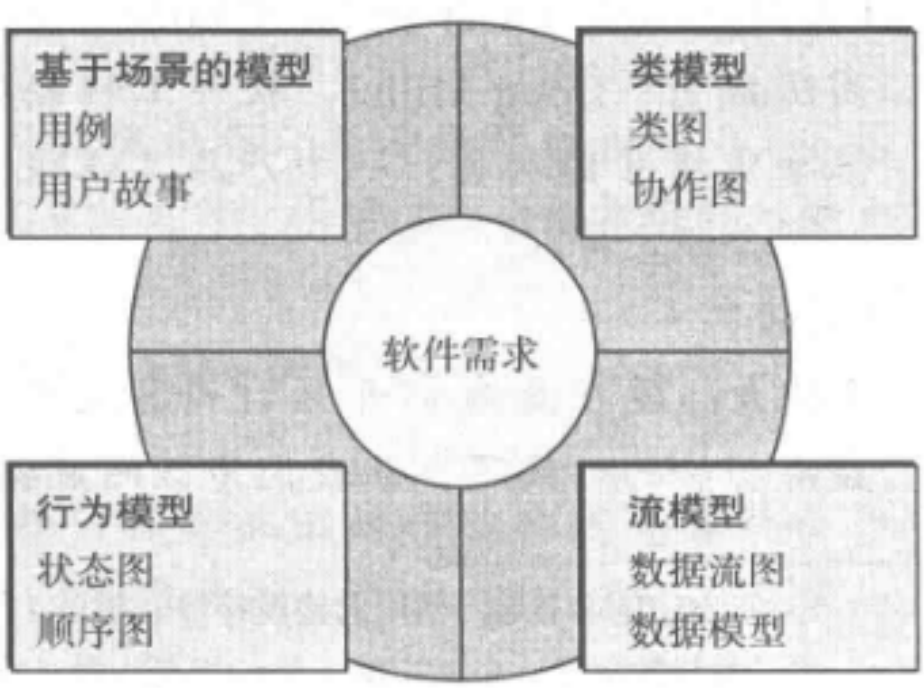
Scenario-base elements depict how the user interacts with the system and specific sequence of activities that occur as the software is used （用例图、活动图）**Use case、**

user story

Class-based elements model the objects that the system will manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined（类图、协作图）**Class diagram、collaboration diagram**

Behavioral elements depict how external events change the state of the system or the classes that reside within it. （状态图、顺序图）**State diagrams、sequence diagram**

Flow-oriented elements represent the system as an information transform, depicting 描绘 how data objects are transformed as they flow through various system functions. （数据流图）**DFD、data models**



- **Scenario-based models** depict requirements from the point of view of various system “actors.”
- **Class-oriented models** represent object-oriented classes (attributes and operations) and how classes collaborate to achieve system requirements.
- **Behavioral models** depict how the software reacts to internal or external “events.”
- **Data models** depict the information domain for the problem.
- **Flow-oriented models** represent functional elements of the system and how they transform data in the system.

> **基于场景的模型**：从各种系统“参与者”的角度描述需求。
> **面向对象的类模型**：表示面向对象的类（属性和操作）以及类如何协作以实现系统需求。
> **行为模型**：描述软件如何响应内部或外部的“事件”。
> **数据模型**：描述问题的信息领域。
> **流模型**：表示系统的功能元素以及它们在系统中如何转换数据。

What is UML?

Diagrams of UML for analysis modeling

Use-case diagram: visualizes the interaction of your system with the outside world

Activity diagram: shows the flow of events within your system

Class diagram: describes the structure of system by showing system's classes, their attributes, and the relationship between the classes

State diagram: indicates system reactions to external events ;represents active states for each class and the events that cause changes between these active states

Swimlane Diagrams: is a useful variation of the activity diagram。 Swimlane diagrams indicate which actor (if there are multiple actors involved in a specif use case) or analysis class has responsibility for the action described by an activity rectangle.

Deployment diagram: Indicates how software functionality and subsystems will be allocated within the physical computing environment

> 用例图：可视化您的系统与外部世界的交互
> 活动图：显示系统内部事件的流动
> 类图：通过显示系统的类、它们的属性以及类之间的关系来描述系统的结构
> 序列图：显示用例中的功能流程
> 协作图：序列图和协作图是同构的
> 状态图：指示系统对外部事件的反应；表示每个类的活动状态以及导致这些活动状态之间变化的事件
> 泳道图：活动图的一种有用变体。泳道图指示哪位参与者（如果特定用例涉及多个参与者）或分析类负责活动矩形所描述的动作
> 部署图：指示在物理计算环境中如何分配软件功能和子系统

Class-Based Modeling 基于类的建模

identifying Analysis Classes

grammatical parse

> 通过检查需求模型(第 8 章)开发的使用场景，并对系统开发的用例进行“语法解析”，我们就可以开始进行类的识别了。带有下划线的每个名词或名词词组可以确定为类，将这些名词输入到一个简单的表中，并标注出同义词。如果要求某个类(名词)实现一个解决方案，那么这个类就是解决方案空间的一部分；否则，如果只要求某个类描述一个解决方案，那么这个类就是问题空间的一部分。

Selecting Classes—Criteria （确定类主要还是开发经验）

Retained information- 保留信息 . The potential class will be useful during analysis only if information about it must be remembered so that the system can function.

Needed services- 所需服务 . The potential class must have a set of identifiable operations that can change the value of its attributes in some way.

Multiple attributes- 多个属性 . During requirement analysis, the focus should be on "major" information; a class with a single attribute may, in fact, be useful during design, but is probably better represented as an attribute of another class during the analysis activity.

Common attributes- 公共属性 . A set of attributes can be defined for the potential class and these attributes apply to all instances of the class.

Common operations- 公共操作 . A set of operations can be defined for the potential class and these operations apply to all instances of the class.

Essential requirements- 必要需求 . External entities that appear in the problem space and produce or consume information essential to the operation of any solution for the system will almost always be defined as classes in the requirements model.

> 选择类的标准（确定类主要还是开发经验）：
> 保留信息：只有在分析中需要记住有关它的信息以使系统能够运行时，潜在的类才是有用的。
> 所需服务：潜在类必须具有一组可识别的操作，这些操作能够以某种方式改变其属性的值。
> 多个属性：在需求分析期间，应关注“主要”信息；在设计阶段，具有单个属性的类可能是有用的，但在分析活动中最好将其表示为另一个类的属性。
> 公共属性：可以为潜在类定义一组属性，这些属性适用于类的所有实例。
> 公共操作：可以为潜在类定义一组操作，这些操作适用于类的所有实例。
> 必要需求：出现在问题空间中的外部实体，并生成或消耗系统操作所需的信息，几乎总是会在需求模型中定义为类。

Class-responsibility-collaborator (CRC) Models

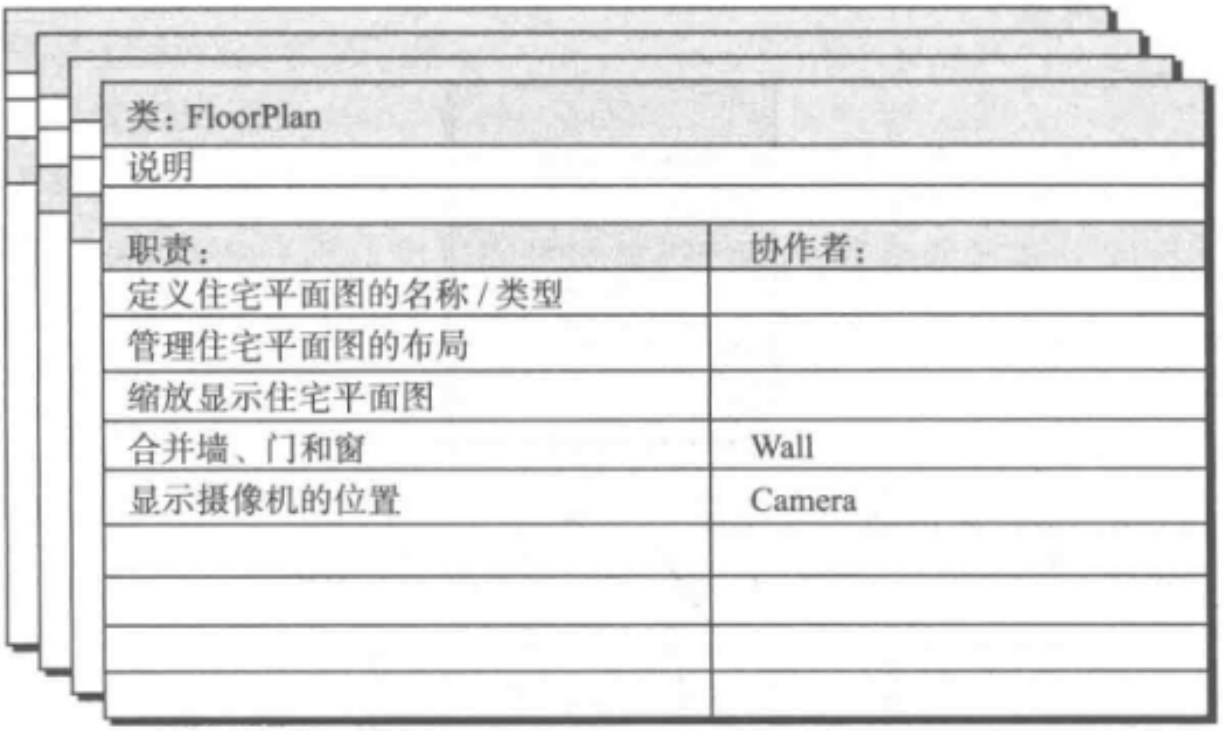
Responsibilities are the attributes and operations encapsulated by the class（封装在类中的属性和操作）

Collaborators are those classes that are required to provide a class with the information needed to complete a responsibility. In general，a collaboration implies either a request for information or a request for some action.（完成某个职责的其它协作的类）

> 职责是和类相关的属性和操作。简单地说，职责就是“类所知道或能做的任何事”
> 协作者是提供完成某个职责所需要信息的类。通常，协作意味着信息请求或某个动作请求。

🔔 CRC 模型

- CRC 模型是表示类的标准索引卡片的集合
- 卡片分为三个部分
- 卡片顶部为类的名称，左边列出了类的职责，右边列出了协作者



类：FloorPlan	
说明	
职责：	协作者：
定义住宅平面图的名称 / 类型	
管理住宅平面图的布局	
缩放显示住宅平面图	
合并墙、门和窗	Wall
显示摄像机的位置	Camera

图 9-3　CRC 模型索引卡

Collaborations

Classes fulfill their responsibilities in one of two ways:

A class can use its own operations to manipulate its own attributes, thereby fulfilling a particular responsibility, or

A class can collaborate with other classes.

Collaborations identify relationships between classes

Collaborations are identified by determining whether a class can fulfill each responsibility itself

Three different generic relationships between classes:

the is-part-of relationship- 聚合

the has-knowledge-of relationship- 关联（常用）

the depends-upon relationship- 依赖

> 类用一种或两种方法来实现其职责:(1)类可以使用其自身的操作控制各自的属性，从而实现特定的职责;(2)类可以和其他类协作。
> 协作识别类之间的关系，协作通过确定一个类是否可以独立履行其职责来识别。
> 类之间有三种不同的通用关系：
> **聚合关系**（is-part-of relationship）
> **关联关系**（has-knowledge-of relationship，常用）
> **依赖关系**（depends-upon relationship）

Design

Design conceptions 设计理念

- Information Hiding 信息隐蔽

> 模块化概念面临的一个基本问题是：“应当如何分解一个软件解决方案以获得最好的模块集合？”信息隐蔽原则建议模块应该“具有的特征是：每个模块对其他所有模块都隐蔽自己的设计决策”。换句话说，模块应该被特别说明并设计，使信息(算法和数据)都包含在模块内，其他模块无需对这些信息进行访问。
> 隐蔽的含义是，通过定义一系列独立的模块得到有效的模块化，独立模块之间只交流实现软件功能所必需的信息。抽象有助于定义构成软件的过程(或信息)实体。隐蔽定义并加强了对模块内过程细节的访问约束以及对模块所使用的任何局部数据结构的访问约束。
> 将信息隐蔽作为系统模块化的一个设计标准，在测试过程以及随后的软件维护过程中需要进行修改时，将使我们受益匪浅。由于大多数数据和过程对软件的其他部分是隐蔽的，因此，在修改过程中不小心引入的错误就不太可能传播到软件的其他地方。
> **每个模块对其它所有模块都隐蔽自己的设计决策。即：在本模块中包含的信息（算法和数据）不要被（不需要这些信息的）其它模块访问（影响）。**
> **封装**
> **尽量少的全局变量**
> **Why Information Hiding?（重要）**
> reduces the likelihood of “side effects”（减少副作用、不良反应的可能性）
> limits the global impact of local design decisions
> emphasizes communication through controlled interfaces
> discourages the use of global data
> leads to encapsulation —an attribute of high quality design
> results in higher quality software
> 减少“副作用”（不良反应）的可能性。
> 限制局部设计决策对整体的影响。
> 强调通过受控接口进行通信。
> 不鼓励使用全局数据。

> 引导封装——高质量设计的一个属性。
> 产生更高质量的软件。

- Separation of concerns 关注点分离

> **关注点分离是一个设计概念，它表明任何复杂问题如果被分解为可以独立解决或优化的若干块，该复杂问题便能够更容易地得到处理。**关注点是一个特征或一个行为，被指定为软件需求模型的一部分。将关注点分割为更小的关注点(由此产生更多可管理的块)，便可用更少的工作量和时间解决一个问题。
> 另一个结果是：两个问题被结合到一起的认知复杂度经常高于每个问题各自的认知复杂度之和。这就引出了“分而治之”的策略 -- 把一个复杂问题分解为若干可管理的块来求解时将会更容易。这对于软件的模块化具有重要的意义。
> 关注点分离在其他相关设计概念中也有体现：模块化、方面、功能独立、求精。每个概念都会在后面的小节中讨论。
> **模块化是关注点分离最常见的表现**

- Modularity(the number and size of modularity) 模块化

> 模块化是关注点分离最常见的表现。软件被划分为独立命名的、可处理的构件，有时被称为模块，把这些构件集成到一起可以满足问题的需求。
> 有人提出“模块化是软件的单一属性，它使程序能被智能化地管理”。软件工程师难以掌握单块软件(即由一个单独模块构成的大程序)。对于单块大型程序，其控制路径的数量、引用的跨度、变量的数量和整体的复杂度使得理解这样的软件几乎是不可能的绝大多数情况下，为了理解更容易，都应当将设计划分成许多模块，这样做的结果是降低构建软件所需的成本。模块化设计(以及由其产生的程序)使开发工作更易于规划；可以定义和交付软件增量；更容易实施变更；能够更有效地开展测试和调试；可以进行长期维护而没有严重的副作用。
> **模块是软件被划分为独立命名的，可处理的构件；模块化就是其尽可能相对独立完成单一功能，做到高内聚、低耦合。同时，模块的划分要适度，模块数量也不是划分越多就好。**

- Refactoring 重构

> 很多敏捷方法(第 5 章)都建议一种重要的设计活动 -- 重构，重构是一种重新组织的技术，可以简化构件的设计(或代码)而无需改变其功能或行为。这样定义重构：“重构是使用这样一种方式改变软件系统的过程：**不改变代码(设计)的外部行为而是改进其内部结构。**”
> 在重构软件时，检查现有设计的冗余性、没有使用的设计元素、低效的或不必要的算法、拙劣的或不恰当的数据结构以及其他设计不足，修改这些不足以获得更好的设计。例如，第一次设计迭代可能得到一个构件，表现出很低的内聚性(即执行三个功能但是相互之间仅有有限的联系)。在深思熟虑之后，设计人员可能决定将原构件重新分解为三个独立的构件其中每个构件都表现出更高的内聚性。结果则是，软件更易于集成、测试与维护。

- Cohesion(functional, layer, communicational) 内聚 **显示某个模块相关功能的强度**

> 一个内聚的设计类具有小的、集中的职责集合，并且专注于使用属性和方法来实现那些职责。例如，视频编辑软件的 Video Clip 类可能包含一组用于编辑视频剪辑的方法。只要每个方法只关注于和视频剪辑相关的属性，内聚性就得以维持。

- Coupling(Content, Control, External) 耦合 **显示模块间相互依赖性**

> 在设计模型内，设计类之间相互协作是必然的。但是，协作应该保持在一个可以接受的最小范围内。如果设计模型高度耦合(每一个设计类都和其他所有的设计类有协作关系)，那么系统就难以实现、测试，并且维护也很费力。通常，一个子系统内的设计类对其他子系统中的类应仅有有限的了解。该限制被称作 Demeter 定律，该定律建议一个方法应该只向周边类中的方法发送消息。

Object-oriented Design conceptions 面向对象的设计理念

Class,attribute, operation

Polymorphism

Encapsulation

Inheritance

> 类，属性，操作
> 多态
> 封装
> 继承

软件质量属性 Quality attributes-FURPS（重点掌握）

Functionality 功能性

Usability 易用性

Reliability 可靠性

Performance- 高性能 processing speed/response time/resource consumption/throughput/efficiency **处理速度 / 响应时间 / 资源消耗 / 吞吐量 / 效率**

Supportability- 可支持性 **（可维护、可测试、兼容、可配置、简易性等）**

maintainability/testability/compatibility/

Configurability-the ability to organize and control elements of the software configuration

Convenient Installation

Locate problems easily

Four Design Models 四种设计模型

Data/Class design – transforms analysis classes into implementation classes and data structures

Architectural design – defines relationships among the major software structural elements

Interface design – defines how software elements, hardware elements, and end-users communicate

Component-level design – transforms structural elements into procedural descriptions of software components

> 数据 / 类设计 – 将分析类转换为实现类和数据结构
> 体系结构设计 – 定义主要软件结构元素之间的关系
> 界面设计 – 定义软件元素、硬件元素和终端用户如何通信
> 组件级设计 – 将结构元素转换为软件组件的过程描述

Architectural Design 体系结构设计

Software Architecture

the structure of the system, which comprise software components, the externally visible properties of those components and the relationships among them.

> 架构包括软件构件，构件的属性和构件之间的相互关系

Architectural description (AD)

The IEEE Standard defines an architectural description (A D) as a “a collection of products to document an architecture.”. An architecture description shall identify the system stakeholders having concerns considered fundamental to the architecture of the system-of-interest.

> 体系结构描述 (AD)：IEEE 标准将体系结构描述定义为“用于记录体系结构的一系列产品”。体系结构描述应识别对目标系统的体系结构具有基本关注的系统利益相关者。

Architectural decision

means that the system architect considers a variety of alternatives and ultimately decides on the specific architectural features that best meet the concern.

> 体系结构决策：意味着系统架构师会考虑各种替代方案，并最终决定最能满足关注点的具体体系结构特征。

Why is Architecture Important?

有助于各方的交流

突出早期的设计，对其后工作很重要（打好设计的根基）

建立了一个相对小、易理解的模型，其描述了系统如何构成及构件如何一起工作

> 体系结构“构建了一个相对小的、易于理解的模型，该模型描述了系统如何构成以及其构件如何一起工作”。
> 软件体系结构提供了一种表示，有助于对计算机系统开发感兴趣的所有利益相关者开展交流。
> 体系结构突出了早期的设计决策，这些决策对随后所有的软件工程工作有深远的影响。

architectural styles

🔔 An architectural style encompasses a set of components, a set of connectors, constraints, and (a) models.

a. semantic b. sentimental c. syntactic d. systematic

描述一组构件，以及它们的连接关系，以及约束条件。

Each style describes a system category that encompasses:

(1) a set of components (e.g., a database, computational modules) that perform a function required by a system, 完成系统需要的某种功能的一组构件

(2) a set of connectors that enable “communication, coordination and cooperation” among components,

能够使构件间实现通信、协作的一组连接件

(3) constraints that define how components can be integrated to form the system, and

定义构建如何成为系统的约束

(4) semantic models 语义模型 that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

文字描述，便于理解

Data-centered architectures

工程学科中是非常普遍的。

Mary Shaw, David Garlan

以数据为中心的体系结构。数据存储（如文件或数据库）位于这种体系结构的中心，其他构件会经常访问该数据存储，并对存储中的数据进行更新、增加、删除或者修改。图 12-1 描述了一种典型的以数据为中心的体系结构风格，其中，客户软件访问中心存储库。在某些情况下，数据存储库是被动的，也就是说，客户软件独立于数据的任何变化或其他客户软件的动作而访问数据。该方法的一个变种是将中心存储库变换成“黑板”，当客户感兴趣的数据发生变化时，它将通知客户软件。

以数据为中心的体系结构促进了可集成性（integrability）[Bas03]，也就是说，现有的构件可以被修改，而且新的客户构件可以加入到体系结构中，而无需考虑其他的客户（因为客户构件是独立运作的）。另外，数据可以在客户间通过“黑板”机制传送（即黑板构件负责协调信息在客户间的传递），客户构件独立地执行过程。

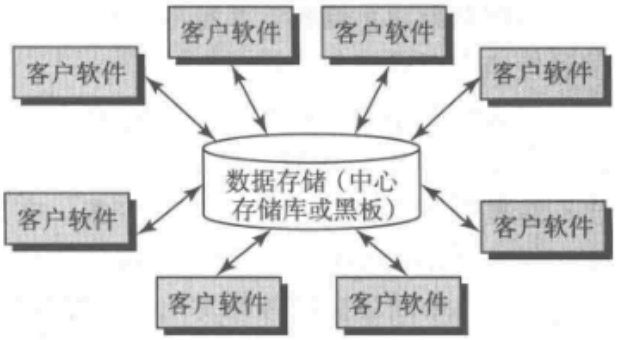


图 12-1　以数据为中心的体系结构

Data flow architectures

数据流体系结构。当输入数据经过一系列计算构件和操作构件的变换形成输出数据时，可以应用这种体系结构。管道 – 过滤器模式（图 12-2）拥有一组称为过滤器的构件，这些构件通过管道连接，管道将数据从一个构件传送到下一个构件。每个过滤器独立于其上游和下游的构件而工作，过滤器的设计要针对某种形式的数据输入，并且产生某种特定形式的数据输出（到下一个过滤器）。然而，过滤器没有必要了解与之相邻的其他过滤器的工作。

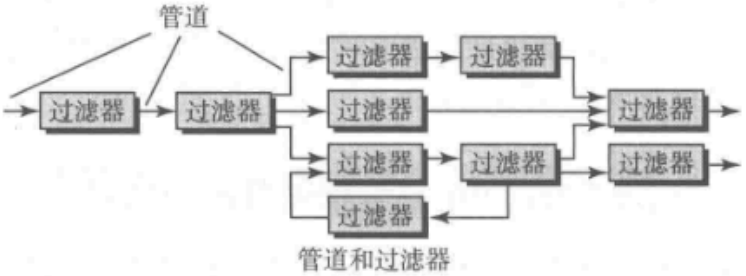


图 12-2　数据流体系结构

如果数据流退化成单线变换，则称为批处理序列（batch sequential）。这种结构接收一批

数据，然后应用一系列连续的构件（过滤器）完成变换。

Call and return architectures

调用和返回体系结构。该体系结构风格能够设计出一个相对易于修改和扩展的程序结构。在此类体系结构中，存在几种子风格 [Bas03]：

- 主程序 / 子程序体系结构。这种传统的程序结构将功能分解为一个控制层次，其中“主”程序调用一组程序构件，这些程序构件又去调用其他构件。图 12-3 描述了该类型的体系结构。
- 远程过程调用体系结构。主程序 / 子程序体系结构的构件分布在网络中的多台计算机上。

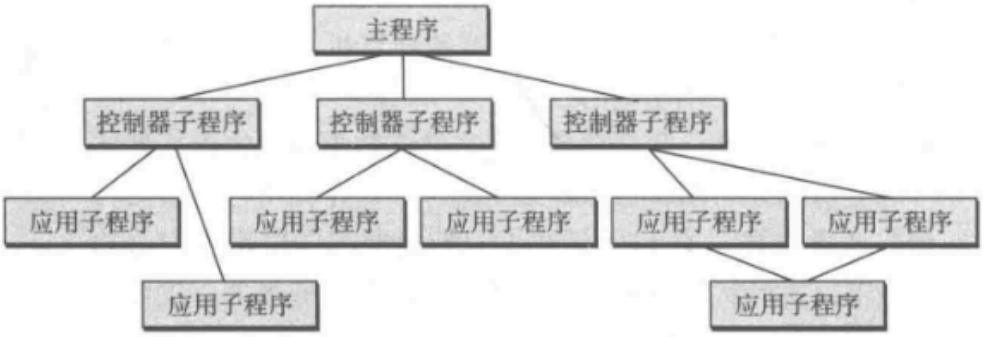


图 12-3　主程序 / 子程序体系结构

Object-oriented architectures

面向对象体系结构。系统的构件封装了数据和必须用于控制该数据的操作，构件间通过信息传递进行通信与合作。

Layered architectures

层次体系结构。层次体系结构的基本结构如图 12-4 所示。其中定义了一系列不同的层次，每个层次各自完成操作，这些操作逐渐接近机器的指令集。在外层，构件完成建立用户界面的操作；在内层，构件完成建立操作系统接口的操作；中间层提供各种实用工具服务和应用软件功能。

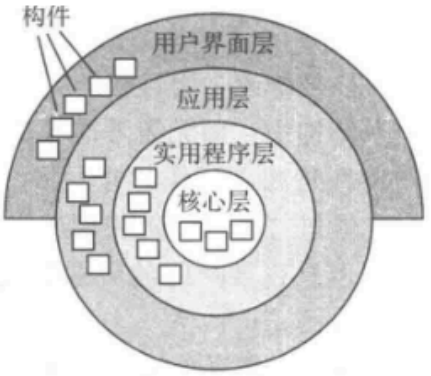


图 12-4　层次体系结构

以上描述的体系结构风格仅仅是可用风格中的一小部分[⊖]。一旦需求工程揭示了待构建系统的特征和约束，就可以选择最适合这些特征和约束的体系结构风格或风格的组合。在很多情况下，会有多种模式是适合的，需要对可选的体系结构风格进行设计和评估。例如，在很多数据库应用中，层次体系结构（适合大多数系统）可以与以数据为中心的体系结构结合起来使用。

确定一个合适的体系结构风格殊为不易，Buschman [Bus10a] 提出的两种补充思想可以对此提供一些指导。问题帧是指在不考虑专业领域知识或程序实现方案细节的前提下，描述反复出现的问题的特征。领域驱动设计则建议软件设计应该反映出你的应用（第 7 章）试图解决的业务问题的领域以及该领域的逻辑。

Component-level Design 构件级设计

Component definition

a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.”

> 系统中模块化的、可部署的和可替换的部件，该部件封装了实现并对外提供一组接口。

What is Component?

O-O View

在面向对象软件工程环境中，构件包括一个协作类集合。构件中的每个类都得到详细阐述，包括所有属性和与其实现相关的操作。作为细节设计的一部分，必须定义所有与其他设计类相互通信协作的接口。为此，设计师需要从分析模型开始，详细描述分析类(对于构件而言该类与问题域相关)和基础类(对于构件而言该类为问题域提供了支持性服务)。

为了说明设计细化过程，考虑为一个高级影印中心构造软件。软件的目的是收集前台的客户需求，对印刷业务进行定价，然后把印刷任务交给自动生产设备。在需求工程中得到了一个名为 PrintJob 的分析类。分析过程中定义的属性和操作在图 13-1 的上方给出了注释在体系结构设计中，PrintJob 被定义为软件体系结构的一个构件，用简化的 UML 符号表示的该构件显示在图 13-1 中部靠右的位置。需要注意的是，PrintJob 有两个接口 :computeJob 和 initiateJob。computeJob 具有对任务进行定价的功能 ,initiateJob 能够把任务传给生产设备这两个接口在图下方的左边给出(即所谓的棒棒糖式符号)。

构件级设计将由此开始。必须对 PrintJob 构件的细节进行细化，以提供指导实现的充分信息。通过不断补充作为构件 PrintJob 的类的全部属性和操作，来逐步细化最初的分析类。正如图 13-1 右下部分的描述，细化后的设计类 PrintJob 包含更多的属性信息和构件实现所需要的更广泛的操作描述。computeJob 和 initiateJob 接口隐含着与其他构件(图中没有显示出来)的通信和协作。例如，computePageCost()操作(computeJob 接口的组成部分)可能与包含任务定价信息的 PricingTable 构件进行协作。checkPriorityO)操作(initiateJob 接口的组成部分)可能与 JobQucue 构件进行协作，用来判断当前等待生产的任务类型和优先级。

对于体系结构设计组成部分的每个构件都要实施细化。细化一旦完成，要对每个属性、每个操作和每个接口进行更进一步的细化。对适合每个属性的数据结构必须予以详细说明。另外还要说明实现与操作相关的处理逻辑的算法细节，在这一章的后半部分将要对这种过程设计活动进行讨论。最后是实现接口所需机制的设计。对于面向对象软件，还包含对实现系统内部对象间消息通信机制的描述。

Conventional view

在传统软件工程环境中，一个构件就是程序的一个功能要素，程序由处理逻辑及实现处理逻辑所需的内部数据结构以及能够保证构件被调用和实现数据传递的接口构成。传统构件也称为模块，作为软件体系结构的一部分，它扮演如下三个重要角色之一 :(1)控制构件协调问题域中所有其他构件的调用 :(2)问题域构件，完成客户需要的全部功能或部分功能 ;(3)基础设施构件，负责完成问题域中所需的支持处理的功能。

与面向对象的构件类似，传统的软件构件也来自于分析模型。不同的是在这种情况下，是以分析模型中的构件细化作为导出构件的基础。构件层次结构上的每个构件都被映射为某一层次上的模块(12.6 节)。一般来讲，控制构件(模块)位于层次结构(体系结构)顶层附近，而问题域构件则倾向位于层次结构的底层。为了获得有效的模块化，在构件细化的过程中采用了功能独立性的设计概念(第 11 章)。

The definition of Decision Table

	Rules					
Conditions	**1**	**2**	**3**	**4**	**5**	**6**
regular customer	T	T				
silver customer			T	T		
gold customer					T	T
special discount	F	T	F	T	F	T
Rules						
no discount	✓					
apply 8 percent discount			✓	✓		
apply 15 percent discount					✓	✓
apply additional x percent discount		✓		✓		✓

分析设计类

Component-level Design (class-based)

The Open-Closed Principle (OCP)

开闭原则（The Open-Closed Principle，OCP）。模块（构件）应该对外延具有开放性，对修改具有封闭性 [Mar00]。这段话似乎有些自相矛盾，但是它却体现出优秀的构件级设计应该具有的最重要特征之一。简单地说，设计者应该采用一种无需对构件自身内部（代码或者内部逻辑）做修改就可以进行扩展（在构件所确定的功能域内）的方式来说明构件。为了达到这个目的，设计者需要进行抽象，在那些可能需要扩展的功能与设计类本身之间起到缓冲区的作用。

例如，假设 SafeHome 的安全功能使用了对各种类型安全传感器进行状态检查的 Detector 类。随着时间的推移，安全传感器的类型和数量将会不断增长。如果内部处理逻辑采用一系列 if-then-else 的结构来实现，其中每个这样的结构都负责一个不同的传感器类型，那么对于新增加的传感器类型，就需要增加额外的内部处理逻辑（依然是另外的 if-then-else 结构），而这显然违背 OCP 原则。

图 13-4 中给出了一种遵循 OCP 原则实现 Detector 类的方法。对于各种不同的传感器，Sensor 接口都向 Detector 构件呈现一致的视图。如果要添加新类型的传感器，那么对 Detector 类（构件）无需进行任何改变。这个设计遵守了 OCP 原则。

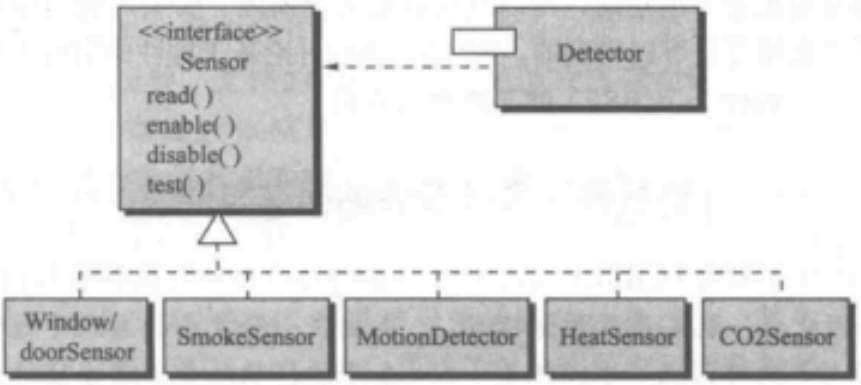


图 13-4　遵循 OCP 原则

The Liskov Substitution Principle (LSP)

Liskov 替换原则（Liskov Substitution Principle，LSP）。子类可以替换它们的基类 [Mar00]。最早提出该设计原则的 Barbara Liskov[Lis88] 建议，将从基类导出的类传递给构件时，使用基类的构件应该仍然能够正确完成其功能。LSP 原则要求源自基类的任何子类必须遵守基类与使用该基类的构件之间的隐含约定。在这里的讨论中，“约定”既是前置条件——构件使用基类前必须为真；又是后置条件——构件使用基类后必须为真。当设计者创建了导出类，则这些子类必须遵守前置条件和后置条件。

Dependency Inversion Principle（DIP）

依赖倒置原则（Dependency Inversion Principle，DIP）。依赖于抽象，而非具体实现 [MarR00]。正如我们在 OCP 中讨论的那样，抽象可以比较容易地对设计进行扩展，又不会导致大量的混乱。构件依赖的其他具体构件（不是依赖抽象类，如接口）越多，扩展起来就越困难。

The Interface Segregation Principle（ISP）

接口分离原则（Interface Segregation Principle，ISP）。多个客户专用接口比一个通用接口要好[Mar00]。多个客户构件使用一个服务器类提供的操作的实例有很多。ISP原则建议设计者应该为每个主要的客户类型都设计一个特定的接口。只有那些与特定客户类型相关的操作才应该出现在该客户的接口说明中。如果多个客户要求相同的操作，则这些操作应该在每个特定的接口中都加以说明。

"具体实现"，你违背了DIP原则。

Interface Design 界面设计

Golden Rules

Place the user in control- 把控制权交给用户

Reduce the user's memory load- 减少用户的记忆负担

Make the interface consistent- 保持界面一致

User Interface Design Models

- **User model** (SE设计开发人员建立用户模型，了解用户并分类)
  a profile of all end users of the system
- **Design model** (SE设计开发人员建立设计模型)
  a design realization of the user model
- **Mental model** (*system perception*) (最终用户在脑海里对系统的想象)
  the user's mental image of what the interface is;
  is the image of system that users carry in their heads.
- **Implementation model** (implementer建立实现模型，就是真实系统的展现)
  the interface "***look and feel***" coupled with supporting information that describe interface syntax and semantics

User Interface Design Process

Framework activities: 分析（建模）- 设计 - 实现 - 确认

- Interface analysis and modeling
  - user analysis

    - task analysis
    - Display Content analysis
    - work environment analysis
- Interface design
    - Using information developed during interface analysis, define interface objects and actions (operations).
    - Define events (user actions) that will cause the state of the user interface to change. Model this behavior
    - Depict each interface state as it will actually look to the end user.
    - Indicate how the user interprets the state of the system from information provided through the interface.
- Interface construction
- Interface validation

> **界面分析与建模**
> - 用户分析
> - 任务分析
> - 显示内容分析
> - 工作环境分析
>
> **界面设计**
> - 使用在界面分析期间开发的信息，定义界面对象和操作（操作）。
> - 定义用户操作事件，这些事件将导致用户界面状态发生变化。建模这种行为。
> - 以最终用户实际看到的方式描述每个界面状态。
> - 指出用户如何从界面提供的信息中解释系统状态。
>
> **界面构建**
>
> **界面验证**

Test

Test

Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.

> 测试的目的是为了发现软件设计和实现过程中的疏忽所造成的错误

> 🔔
> - 软件测试的目标是尽可能早地找到软件缺陷，并确保其得以修复

- 使用人工或自动地手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的需求或弄清预期结果与实际结果之间的差距
- 测试是以评价一个程序或者系统属性为目标的任何一种活动，测试是对软件质量的度量

Test Strategies 测试策略

To perform effective testing, you should conduct effective technical reviews. By doing this, many errors will be eliminated before testing commences

> 为了执行有效的测试，您应该进行有效的技术审查。这样做可以在测试开始前消除许多错误

Testing begins at the component level and works "outward" toward the integration of the entire computer-based system.

> 测试从组件级别开始，并“向外”工作，以集成整个基于计算机的系统。

Different testing techniques are appropriate for different software engineering approaches and at different points in time.

> 不同的测试技术适用于不同的软件工程方法和不同的时间点。

Testing is conducted by the developer of the software and (for large projects) an independent test group.

> 测试是由软件开发人员和（对于大型项目）一个独立的测试小组进行的。

Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

> 测试和调试是不同的活动，但是调试必须包含在任何测试策略中。

Test Strategy 测试策略

Unit testing begins at the center of the spiral and concentrates on each unit (for example, component, class, or content object) as they are implemented in source code.

> 单元测试从螺旋的中心开始，集中在源代码中实现的每个单元（例如，组件、类或内容对象）上。

Testing progresses to integration testing, where the focus is on design and the construction of the software architecture. Taking another turn outward on the spiral.

> 测试发展到集成测试，重点放在软件架构的设计和构造上。再向外旋转一次。

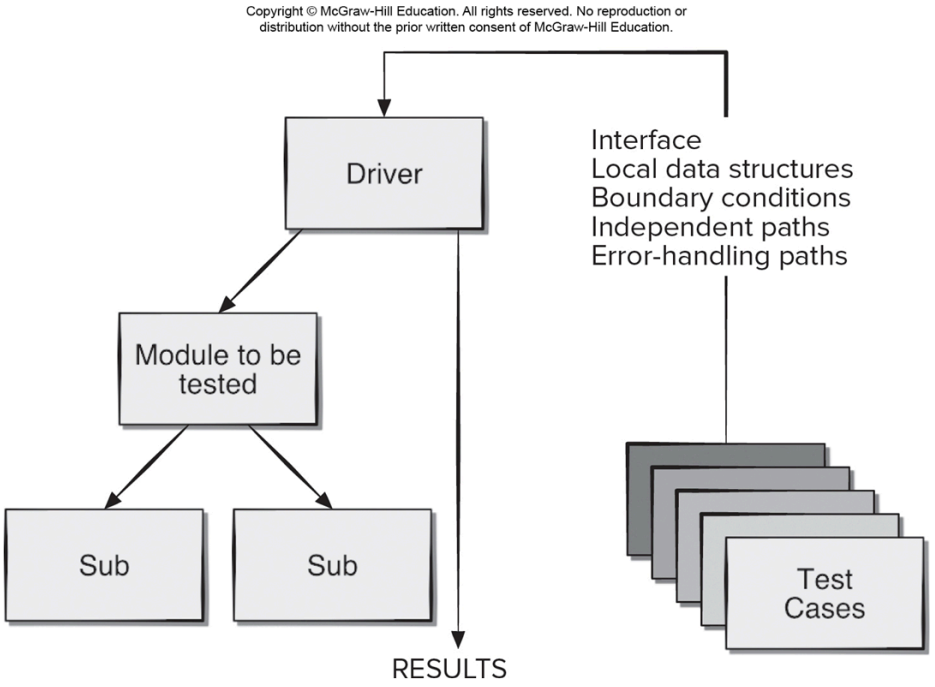
Validation testing, is where requirements established as part of requirements modeling are validated against the software that has been constructed.

> 验证测试是根据已经构建的软件验证作为需求建模一部分的需求。

In system testing, the software and other system elements are tested as a whole.

> 在系统测试中，软件和其他系统元素作为一个整体进行测试。

Unit Test 单元测试



单元测试：就是构件（模块）测试，侧重于构件的内部处理逻辑和数据结构

A driver is nothing more than a "main program" that accepts test-case data, passes such data to the component (to be tested), and prints relevant results.

> 驱动程序：是一个主程序，它接收测试用例数据，并将这些数据传递给被测模块；

Stubs (dummy subprogram) serve to replace modules invoked by the component to be tested.

> 桩程序的作用是替换那些从属于被测模块的模块

Integration Test 集成测试

Integration testing is a systematic technique for constructing the software architecture while conducting tests to uncover errors associated with interfacing.

> 集成测试是构建软件体系结构的系统化技术，同时也是进行一些旨在发现与接口相关的错误的测试。

Top-down Integration 自顶向下集成

步骤

1. 以主控模块作为测试驱动模块，把对主控模块进行单元测试时引入的所有桩模块用实际模块替代；
2. 依据所选的集成策略（深度优先或广度优先），每次只替代一个桩模块；
3. 每集成一个模块立即测试一遍；
4. 只有每组测试完成后，才着手替换下一个桩模块；
5. 为避免引入新错误，须不断地进行回归测试（即全部或部分地重复已做过的测试）。

优点

1. 在测试过程的早期验证主要的控制点（这些点一般在层次结构的较高层）

2. 若选择深度优先，可以实现和展示软件的某个完整功能，较早的功能展示可以增强开发者和用户的信心

缺点

The major disadvantages of top-down is the need for stubs and the attendant testing difficulties that can be associated with them top-down

缺点：需要桩以及桩所带来的测试困难

Bottom-up Integration 自底向上集成

步骤

1. 把低层模块组织成实现某个子功能的模块群（cluster）；
2. 开发一个测试驱动模块，控制测试数据的输入和测试结果的输出；
3. 对每个模块群进行测试；
4. 删除测试使用的驱动模块，用较高层模块把模块群组织成为完成更大功能的新模块群。

缺点

The major disadvantage of bottom-up is the program as an entity does not exist until the last module is added. This drawback is tempered by easier test case design and a lack of tubs – bottom-up

缺点：需要开发驱动模块，直到最后一个模块加入，才看到一个整体的程序（当然测试用例设计要简单些，也不需要桩程序）

Sandwich Testing 组合方法

用自顶向下的方法测试程序结构的较高层；

用自底向上的方法测试其从属层，比较好的折中方法

Validation Test 确认测试

Software validation is achieved through a series of tests that demonstrate conformity with requirements. A test plan and test cases that are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all content is accurate and properly presented,all performance requirements are attained, documentation is correct, and usability and other requirements are met.

> 软件确认是通过一系列表明软件功能与软件需求相符合的测试而获得的。测试计划列出将要执行的测试类，测试规程定义了特定的测试用例，设计的特定测试用例用于确保软件满足所有的功能需求，具有所有的行为特征，所有内容都准确无误且正确显示，达到所有的性能需求，文档是正确的、可用的，且满足其他需求

Configuration Review 配置评审

配置评审，确保所有软件配置元素已正确开发、编码

Acceptance testing 验收测试

Focus is on customer usage

Alpha testing（工厂测试）：由有代表性的最终用户在工厂进行

Beta testing（用户现场测试）：在一个或多个用户现场（多个环境下）进行（开发者通常不在场）

两者的主要区别是测试的场所不同。一般地，alpha 测试先于 beta 测试执行。通用的软件产品需要较大规模的 beta 测试，测试周期比较长。如果产品通过了 beta 测试，那么就可以正式发行了 （发行 beta 版）

System Test 系统测试

Recovery testing 恢复测试

forces the software to fail in a variety of ways and verifies that recovery is properly performed (系统自动恢复，或人工干预恢复) 极端情况：断电、断网

> 强制软件以各种方式失败，并验证恢复是否正确执行

Security testing 安全测试

verifies that protection mechanisms built into a system will, in fact, protect it from improper penetration（黑客入侵等）

> 验证内置于系统中的保护机制实际上可以保护系统免受不适当的渗透。

Stress testing 压力测试

executes a system in a manner that demands resources in abnormal quantity, frequency, or volume

> 以异常的数量、频率或卷方式执行系统

Performance Testing 性能测试

test the run-time performance of software within the context of an integrated system （比如响应时间或其它性能的问题进行测试）

> 在集成系统的上下文中测试软件的运行时性能

常常与压力测试一同进行，测试系统资源的利用情况，比如：CPU 占有率，内存消耗 - 泄漏等等

Deployment Testing/Configuration testing 部署测试 / 配置测试

Exercises the software in each environment in which it is to operate.

> 在软件将要运行的每个环境中对其进行测试。

Examines all installation procedures and all documentation

> 检查所有安装程序和所有文档。

Test Tactics 测试方法

White-box Test 白盒测试

Sometimes called glass-box testing, is a test-case design philosophy that uses the control structure described as part of component-level design to derive test cases.

> 白盒测试，也叫玻璃盒测试，是一种测试用例设计方法。它利用作为构件层设计的一部分所描述的控制结构来生成测试用例
> 白盒测试是基于过程细节的封闭测试。测试构建内部的数据结构、算法流程与接口。（内部测试）

Basic Path Testing 基本路径测试

基本路径测试是一种白盒测试技术

保证程序每条语句至少执行一次

Control Structure Testing 控制结构测试

Condition testing is a test-case design method that exercises the logical conditions contained in a program module.

> 条件测试是一种测试用例设计方法，它执行程序模块中包含的逻辑条件。

Data flow testing selects test paths of a program according to the locations of definitions and uses of variables in the program.

> 数据流测试根据程序中变量的定义和使用的位置选择程序的测试路径。

Loop testing is a white-box testing technique that focuses exclusively on the validity of loop constructs.

> 循环测试是一种专门关注循环结构有效性的白盒测试技术。

Black-box Test 黑盒测试

Black-box testing, also called behavioral testing, focus on the functional requirements of the software

> 就是功能测试，从外部执行测试用例，用以验证待测功能的正确性，而不考虑软件内部处理逻辑。外部测试

Equivalence Partitioning 等价类划分

等价类划分法是一种典型的、重要的黑盒测试方法，它将程序所有可能的输入数据（有效的和无效的）划分成若干个等价类。然后从每个部分中选取具有代表性的数据当做测试用例进行合理的分类，测试用例由有效等价类和无效等价类的代表组成，从而保证测试用例具有完整性和代表性。

1. 如果输入条件代表一个范围，可以定义一个有效等价类和两个无效等价类（小于最小，大于最大）。
2. 如果输入条件需要特定的值，可以定义一个有效等价类和两个无效等价类。（小于特定值，大于特定值）。
3. 如果输入条件代表集合的某个元素，可以定义一个有效等价类和一个无效等价类。
4. 如果输入条件是布尔式，可以定义一个有效等价类和一个无效等价类。

Boundary Value Analysis 边界值分析

边界值分析也是一种黑盒测试方法，是对“等价划分”的补充，BVA 不是选择等价类的任何元素，而是在等价类“边缘”上选择测试用例。

1. 如果输入条件代表以 a 和 b 为边界的范围，测试用例应当包含 a、b、略大于 a 和略小于 b 的值。
2. 如果输入条件代表一组值，测试用例应当执行其中的最大值和最小值，还应当测试略大于最小值的值和略小于最大值的值。
3. 指导原则 1 和 2 也适用于输出条件，例如，工程分析程序要求输出温度和压强的对照表，测试用例应当能够创建包含最大值和最小值的项。
4. 如果程序数据结构有预定义的边界(如数组有 100 项)，要测试其边界的数据项。

Other 其他

Regression testing

Smoke testing

Debug, Debugging Techniques

OO Test 面向对象测试

OO Unit Testing 面向对象单元测试

the concept of the unit changes, the smallest testable unit is the encapsulated class, a single operation can no longer be tested in isolation (the conventional view of unit testing) but rather, as part of a class

> 单元的概念发生了变化，最小的可测试单元是封装的类，单个操作不能再被孤立地测试（传统的单元测试观点），而是作为类的一部分

OO Integration Testing 面向对象集成测试

Cluster testing is one step in the integration testing of OO software.defines a cluster of collaborating classes (determined hv examining the CRC and object-relationship model) is exercised designing test cases that attempt to uncover errors in the collaborations.

> 集群测试是 OO 软件集成测试中的一个步骤，定义协作类的集群（通过检查 CRC 和对象关系模型确定），并设计测试用例来尝试发现协作中的错误。

Thread-based testing integrates the set of classes required to respond to one input or event for the system

> 基于线程的测试将一组所需的类组合在一起，以响应系统中的一个输入或事件。

Use-based testing begins the construction of the system by testing those classes (called independent classes) that use very few (if any) of server classes. After the independent

classes are tested, the next layer of classes, called dependent classes

> 基于使用的测试是从测试那些使用极少（如果有的话）服务器类的类（称为独立类）开始构建系统。 在独立类测试完成后，接下来测试下一层类，称为依赖类。

Cyclomatic Complexity 环复杂度

$$V(G) = P + 1$$

RationalRose 打开文档

暂不支持查看预览视图